

EX.NO.8a	JAVASCRIPT DOM (HOVER AND CLICKS ON A TABLE)
DATE: 07.09.26	

AIM:

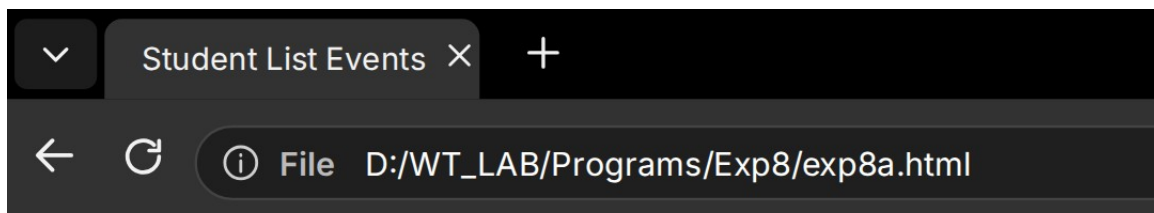
To build a student list in HTML and use JavaScript mouse events (mouseover, mouseout, click and dblclick) to highlight, select and clear table rows dynamically.

ALGORITHM:

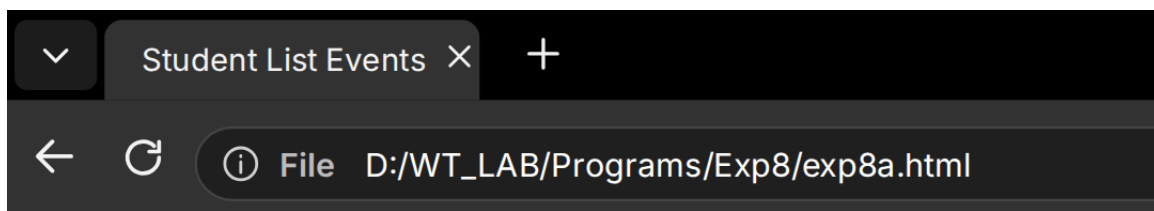
1. Design an HTML table that lists student names, departments and years.
2. Attach a mouseover handler that gives the row under the pointer a yellow background.
3. Attach a mouseout handler that removes the yellow background unless the row is selected.
4. Attach a click handler that marks the row as selected with an orange background.
5. Attach a dblclick handler that clears the selection of that row.
6. Open the page in the browser and observe that the rows react without reloading the page.

CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Student List Events</title>
  <style>
    table { border-collapse: collapse; width: 560px; font-family: Arial; }
    th, td { border: 1px solid #333; padding: 9px 12px; text-align: left; }
    th { background: #cfe3f1; }
    .hover { background: #fff3a8; }
    .picked { background: #ffb36b; font-weight: bold; }
  </style>
</head>
<body>
  <h2>Student List - Mouse Events</h2>
  <table id="stuTable">
    <tr><th>Name</th><th>Department</th><th>Year</th></tr>
    <tr><td>Akhilesh Raj P</td><td>CSE</td><td>III</td></tr>
    <tr><td>Naveen R</td><td>IT</td><td>III</td></tr>
    <tr><td>Divya S</td><td>ECE</td><td>II</td></tr>
    <tr><td>Karthik M</td><td>MECH</td><td>IV</td></tr>
  </table>
  <script>
    var rows = document.querySelectorAll("#stuTable tr");
    for (var i = 1; i < rows.length; i++) {
      rows[i].addEventListener("mouseover", function () { this.classList.add("hover"); });
      rows[i].addEventListener("mouseout", function () { this.classList.remove("hover"); });
    };
    rows[i].addEventListener("click", function () { this.classList.add("picked"); });
    rows[i].addEventListener("dblclick", function () { this.classList.remove("picked"); });
  </script>
</body>
</html>
```

OUTPUT:**Before any event****Student List - Mouse Events**

Name	Department	Year
Akhilesh Raj P	CSE	III
Naveen R	IT	III
Divya S	ECE	II
Karthik M	MECH	IV

After hover, click and double-click**Student List - Mouse Events**

Name	Department	Year
Akhilesh Raj P	CSE	III
Naveen R	IT	III
Divya S	ECE	II
Karthik M	MECH	IV

RESULT:

Thus, the JavaScript mouse events mouseover, mouseout, click and dblclick were applied to the table rows successfully.

EX.NO.8b	JAVASCRIPT DOM (REAL-TIME DIGITAL CLOCK)
DATE: 07.09.26	

AIM:

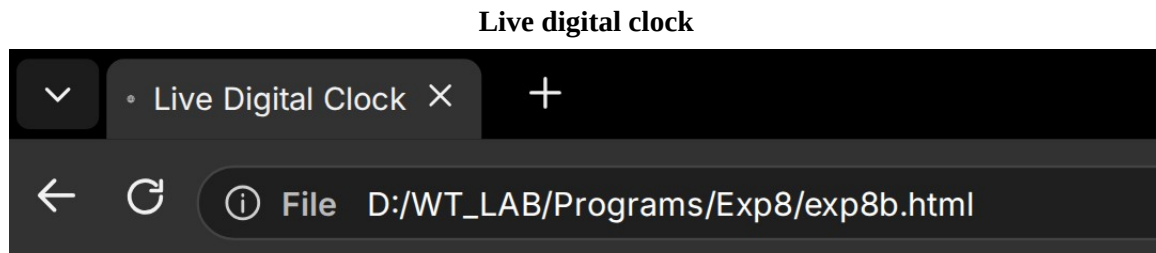
To create a digital clock web page that shows the current date and the live time in 12-hour format using JavaScript timers.

ALGORITHM:

1. Create a page with two elements, one for the date and one for the time.
2. Write a function that reads the current Date object and formats hours, minutes and seconds.
3. Convert the hours to the 12-hour format and add the AM or PM label.
4. Pad single-digit values with a leading zero.
5. Call the function once and then repeat it every second with setInterval().
6. Run the page and confirm that the time changes every second.

CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Live Digital Clock</title>
  <style>
    body { font-family: Arial; text-align: center; margin-top: 70px; }
    #date { font-size: 20px; color: #444; }
    #time { font-size: 56px; font-weight: bold; color: #0b3d91; letter-spacing: 3px; }
  </style>
</head>
<body>
  <h2>Live Digital Clock</h2>
  <div id="date"></div>
  <div id="time"></div>
  <script>
    function two(n) { return n < 10 ? "0" + n : n; }
    function tick() {
      var d = new Date();
      var h = d.getHours();
      var ap = h >= 12 ? "PM" : "AM";
      h = h % 12 || 12;
      document.getElementById("time").innerHTML =
        two(h) + ":" + two(d.getMinutes()) + ":" + two(d.getSeconds()) + " " + ap;
      document.getElementById("date").innerHTML = d.toString();
    }
    tick();
    setInterval(tick, 1000);
  </script>
</body>
</html>
```

OUTPUT:

Live Digital Clock
Mon Sep 07 2026
10:42:18 AM

RESULT:

Thus, a real-time digital clock was created using JavaScript and it updates every second.

EX.NO.8c	JAVASCRIPT DOM (KEYBOARD INTERACTION)
DATE: 07.09.26	

AIM:

To handle keyboard events in JavaScript so that pressing specific keys changes the border style of a table.

ALGORITHM:

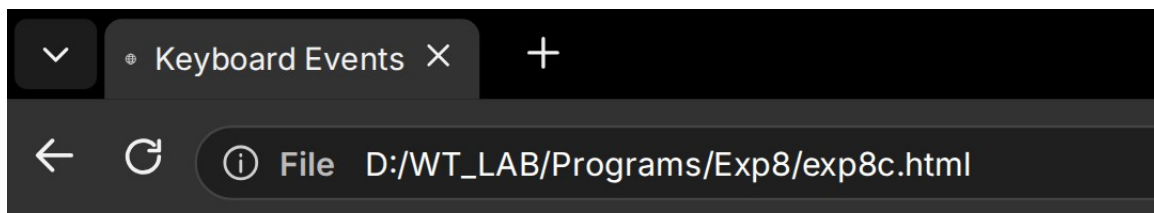
1. Create a page with a short instruction and a small marks table.
2. Register a keydown listener on the document object.
3. If the key G is pressed, give the table a thick green border.
4. If the key R is pressed, remove the border from the table.
5. Display the last pressed key below the table.
6. Press the keys in the browser and check the result.

CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Keyboard Events</title>
  <style>
    body { font-family: Arial; margin: 30px; }
    table { border-collapse: collapse; width: 420px; }
    th, td { border: 1px solid #555; padding: 9px; }
  </style>
</head>
<body>
  <h2>Keyboard Interaction</h2>
  <p>Press G for a green border</p>
  <p>Press R to remove the border</p>
  <table id="mark">
    <tr><th>Subject</th><th>Marks</th></tr>
    <tr><td>Web Technologies</td><td>92</td></tr>
    <tr><td>Microcontrollers</td><td>88</td></tr>
  </table>
  <p id="msg"></p>
  <script>
    var t = document.getElementById("mark");
    document.addEventListener("keydown", function (e) {
      var k = e.key.toUpperCase();
      if (k === "G") t.style.border = "4px solid green";
      if (k === "R") t.style.border = "";
      document.getElementById("msg").innerHTML = "Last key pressed: " + k;
    });
  </script>
</body>
</html>
```

OUTPUT:

Before pressing a key

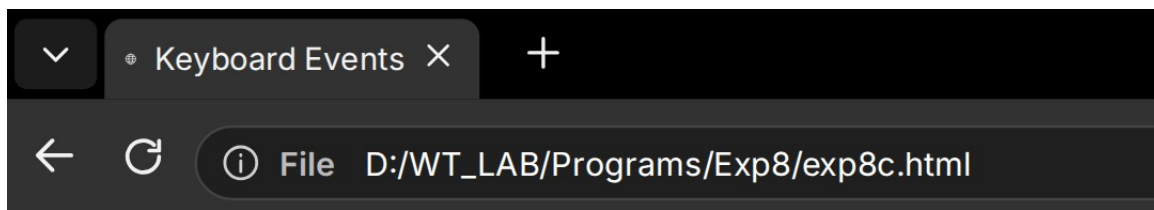
**Keyboard Interaction**

Press G for a green border

Press R to remove the border

Subject	Marks
Web Technologies	92
Microcontrollers	88

After pressing the G key

**Keyboard Interaction**

Press G for a green border

Press R to remove the border

Subject	Marks
Web Technologies	92
Microcontrollers	88

Last key pressed: G

RESULT:

Thus, keyboard events were handled with JavaScript to change the style of the table dynamically.

EX.NO.8d	JAVASCRIPT DOM (SIMPLE FORM WITH DISPLAY)
DATE: 07.09.26	

AIM:

To read a value typed into an HTML form and display a personalised greeting on the same page using JavaScript DOM methods.

ALGORITHM:

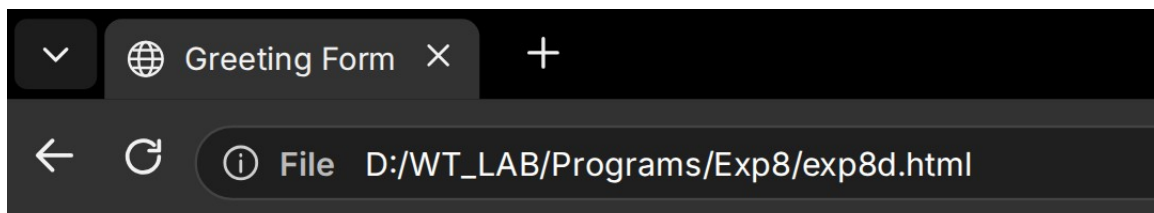
1. Create a form with a text box and a Greet button.
2. Write a function that reads the value of the text box with getElementById().
3. Check whether the box is empty and show a warning if it is.
4. Otherwise build a greeting message from the entered name.
5. Write the message into a paragraph using innerHTML.
6. Enter a name in the browser and verify the displayed greeting.

CODE:

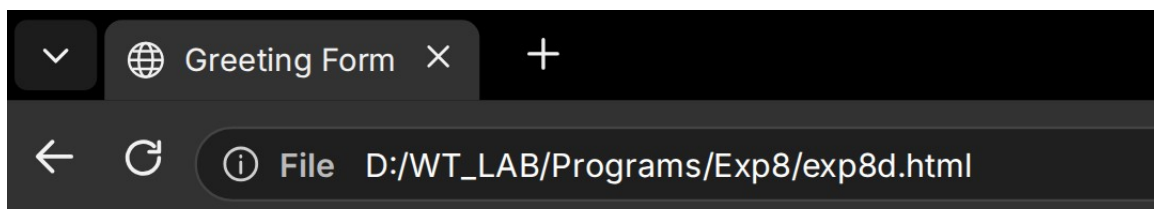
```
<!DOCTYPE html>
<html>
<head>
  <title>Greeting Form</title>
  <style>
    body { font-family: Arial; margin: 40px; }
    input, button { font-size: 16px; padding: 8px 12px; }
    #out { margin-top: 20px; font-size: 20px; color: #0a6b2d; font-weight: bold; }
  </style>
</head>
<body>
  <h2>Greeting Form</h2>
  <input type="text" id="nm" placeholder="Enter your name">
  <button onclick="greet()">Greet</button>
  <p id="out"></p>
  <script>
    function greet() {
      var n = document.getElementById("nm").value.trim();
      var o = document.getElementById("out");
      if (n === "") {
        o.style.color = "red";
        o.innerHTML = "Please enter your name.";
      }
      else {
        o.style.color = "#0a6b2d";
        o.innerHTML = "Hello, " + n + "! Welcome to the Web Technologies Lab.";
      }
    }
  </script>
</body>
</html>
```

OUTPUT:

Empty form

**Greeting Form**

Greeting displayed after clicking Greet

**Greeting Form** **Hello, Akhilesh Raj P! Welcome to the Web Technologies Lab.****RESULT:**

Thus, a simple form was created and the entered name was displayed as a greeting using JavaScript.

EX.NO.8e	JAVASCRIPT DOM (MINI LOGIN SYSTEM)
DATE: 07.09.26	

AIM:

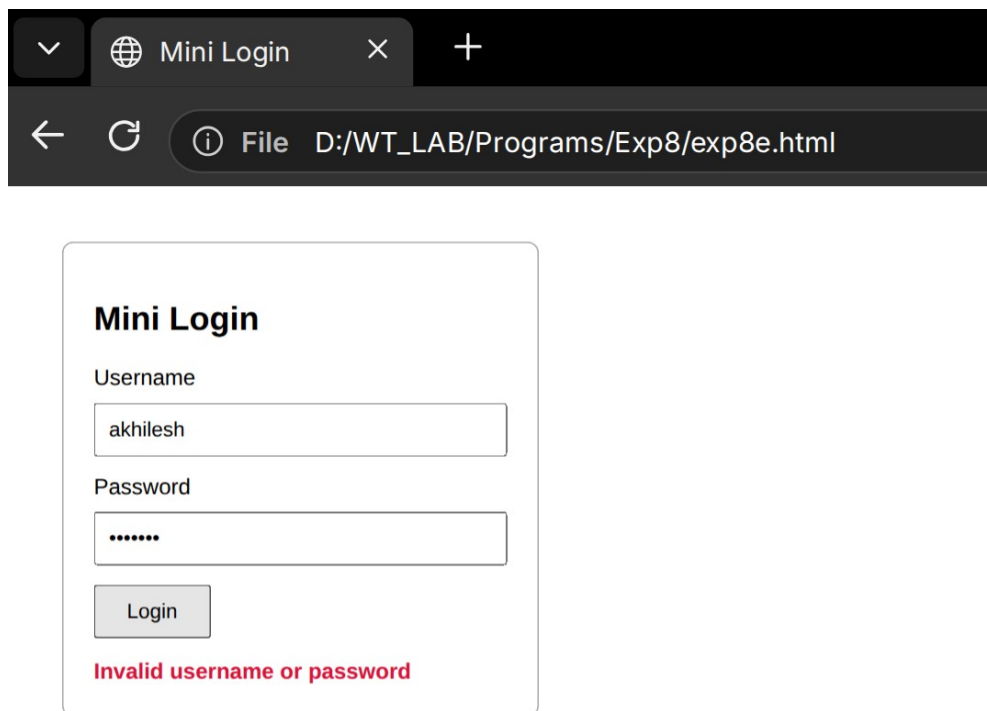
To design a mini login page that validates a username and password with JavaScript and shows a success or failure message.

ALGORITHM:

1. Design a login form with a username box, a password box and a Login button.
2. Store the valid username and password in JavaScript variables.
3. Read the values typed by the user when the button is clicked.
4. Compare the typed values with the stored credentials.
5. Show a welcome message for correct values and an error message otherwise.
6. Test the page with correct and wrong credentials.

CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Mini Login</title>
  <style>
    body { font-family: Arial; margin: 40px; }
    .box { width: 300px; padding: 22px; border: 1px solid #aaa;
          border-radius: 8px; }
    input { width: 100%; padding: 9px; margin: 8px 0 14px;
           font-size: 15px; box-sizing: border-box; }
    button { padding: 9px 22px; font-size: 15px; }
    #res { margin-top: 16px; font-weight: bold; }
  </style>
</head>
<body>
  <div class="box">
    <h2>Mini Login</h2>
    <label>Username</label>
    <input type="text" id="u">
    <label>Password</label>
    <input type="password" id="p">
    <button onclick="check()">Login</button>
    <div id="res"></div>
  </div>
  <script>
    function check() {
      var ok = (document.getElementById("u").value === "akhilesh" &&
                document.getElementById("p").value === "wt@1009");
      var r = document.getElementById("res");
      r.style.color = ok ? "green" : "crimson";
      r.innerHTML = ok ? "Welcome, Akhilesh!" : "Invalid username or password";
    }
  </script>
</body>
</html>
```

OUTPUT:**Wrong password**

Mini Login

Username

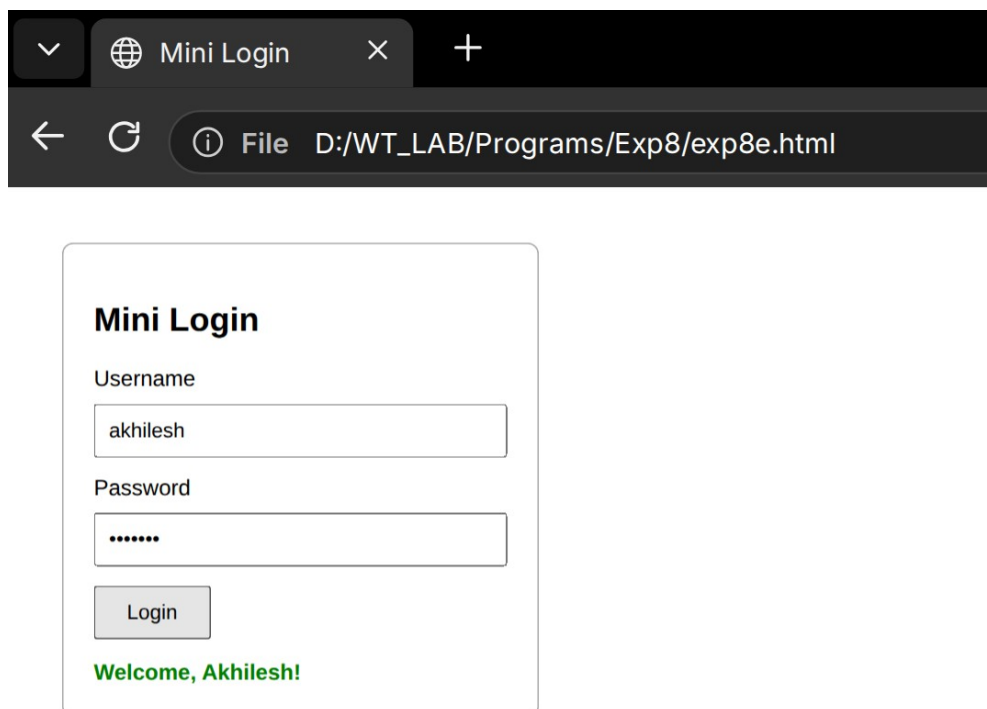
akhilesh

Password

.....

Login

Invalid username or password

Correct credentials

Mini Login

Username

akhilesh

Password

.....

Login

Welcome, Akhilesh!

RESULT:

Thus, a mini login system was implemented and the credentials were validated using JavaScript.

EX.NO.8f	JAVASCRIPT DOM (DARK/LIGHT MODE)
DATE: 07.09.26	

AIM:

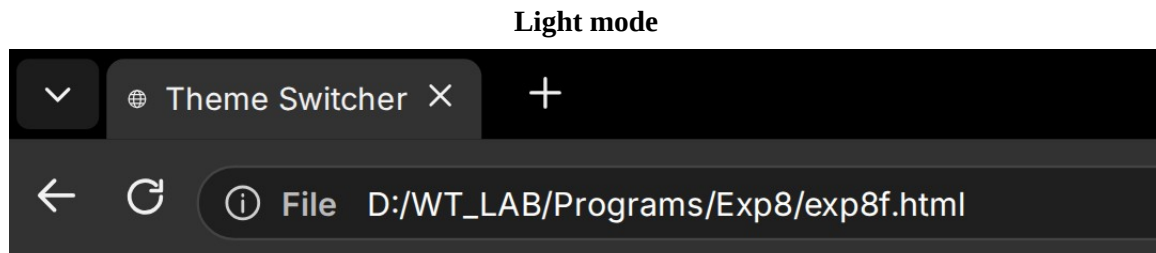
To create a web page with a theme switch button that toggles between a light mode and a dark mode using JavaScript and CSS classes.

ALGORITHM:

1. Define CSS rules for the default light theme and a dark theme class.
2. Add a button that triggers the theme change.
3. Write a function that toggles the dark class on the body element.
4. Change the button text according to the current theme.
5. Save the choice so the page remembers it.
6. Click the button in the browser and check both themes.

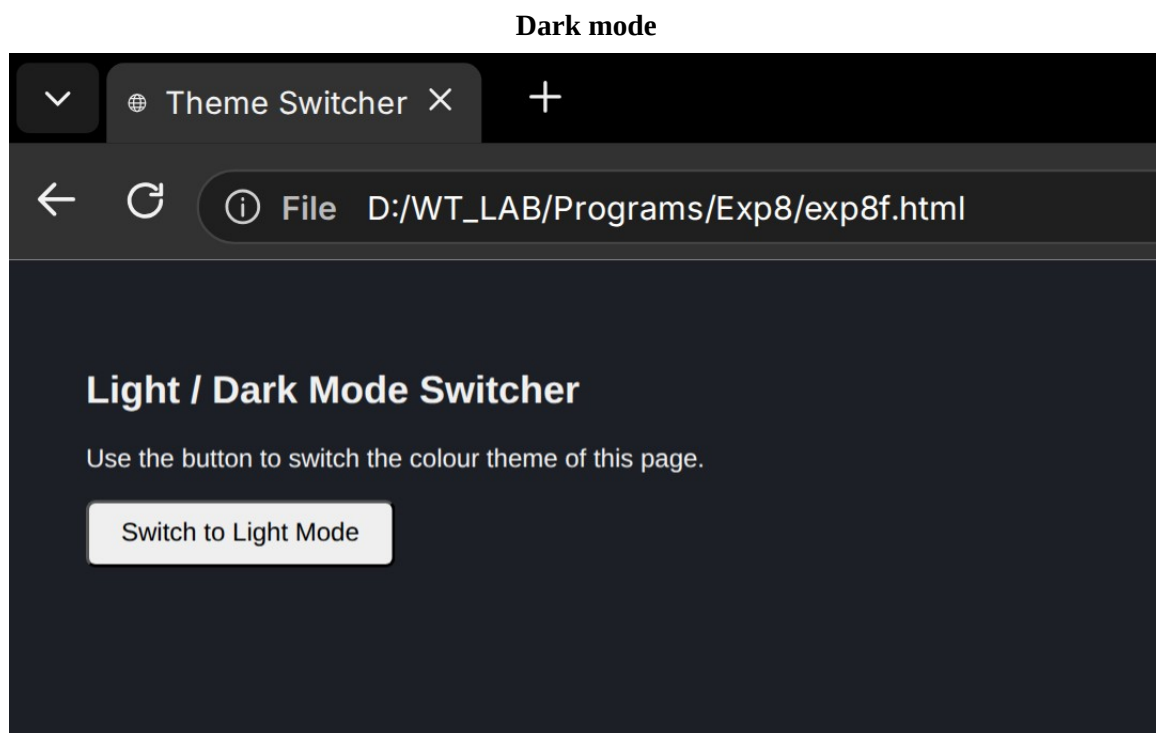
CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Theme Switcher</title>
  <style>
    body { font-family: Arial; background: #ffffff; color: #1b1b1b;
      padding: 40px; transition: .3s; }
    body.dark { background: #1c1f26; color: #f1f1f1; }
    button { padding: 10px 20px; font-size: 16px; cursor: pointer; border-radius: 6px; }
  </style>
</head>
<body>
  <h2>Light / Dark Mode Switcher</h2>
  <p>Use the button to switch the colour theme of this page.</p>
  <button id="tb" onclick="swap()">Switch to Dark Mode</button>
  <script>
    function swap() {
      var dark = document.body.classList.toggle("dark");
      document.getElementById("tb").innerHTML =
        dark ? "Switch to Light Mode" : "Switch to Dark Mode";
      localStorage.setItem("theme", dark ? "dark" : "light");
    }
  </script>
</body>
</html>
```

OUTPUT:**Light / Dark Mode Switcher**

Use the button to switch the colour theme of this page.

Switch to Dark Mode

**RESULT:**

Thus, a dark and light mode switcher was developed using JavaScript DOM manipulation.

EX.NO.9	ANGULAR JS FORM VALIDATION
DATE: 21.09.26	

AIM:

To build a course enrolment form in AngularJS and validate the name, email, age and mobile number fields using the built-in form validation directives.

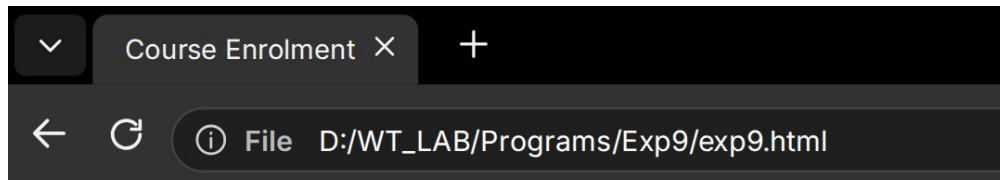
ALGORITHM:

1. Include the AngularJS library and create a module and controller named enrolApp and enrolCtrl.
2. Create a form with the name enrolForm and set novalidate on it.
3. Add required and ng-minlength validation to the name field.
4. Use type="email" for the email field and min and max limits for the age field.
5. Use ng-pattern to accept exactly ten digits for the mobile number.
6. Show error messages with ng-show using \$dirty and \$error.
7. Disable the Enrol button until the whole form is valid (\$invalid).
8. Display the entered details after a successful submit.

CODE:

```
<!DOCTYPE html>
<html ng-app="enrolApp">
<head>
  <title>Course Enrolment</title>
  <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>
  <style>
    body { font-family: Arial; margin: 30px; }
    .err { color: crimson; font-size: 13px; }
    input.ng-invalid.ng-dirty { border: 2px solid crimson; }
    input.ng-valid.ng-dirty { border: 2px solid green; }
  </style>
</head>
<body ng-controller="enrolCtrl">
  <h2>Course Enrolment Form</h2>
  <form name="enrolForm" novalidate ng-submit="save()">
    <p>Name:<br>
      <input type="text" name="nm" ng-model="s.name" required ng-minlength="3">
      <span class="err" ng-show="enrolForm.nm.$dirty && enrolForm.nm.$invalid">Name must have at least 3 letters</span></p>
    <p>Email:<br>
      <input type="email" name="em" ng-model="s.email" required>
      <span class="err" ng-show="enrolForm.em.$dirty && enrolForm.em.$invalid">Enter a valid email</span></p>
    <p>Age:<br>
      <input type="number" name="ag" ng-model="s.age" min="17" max="30" required>
      <span class="err" ng-show="enrolForm.ag.$dirty && enrolForm.ag.$invalid">Age must be between 17 and 30</span></p>
    <p>Mobile:<br>
      <input type="text" name="mb" ng-model="s.mobile" ng-pattern="/^[0-9]{10}$/" required>
      <span class="err" ng-show="enrolForm.mb.$dirty && enrolForm.mb.$invalid">Mobile must have 10 digits</span></p>
    <button type="submit" ng-disabled="enrolForm.$invalid">Enrol</button>
  </form>
  <p ng-show="done"><b>Enrolled:</b> {{s.name}}, {{s.email}}, {{s.age}}, {{s.mobile}}</p>
</body>
<script>
  angular.module("enrolApp", []).controller("enrolCtrl", function ($scope) {
    $scope.s = {};
  });
</script>
```

```
$scope.done = false;  
$scope.save = function () { $scope.done = true; };  
});  
</script>  
</body>  
</html>
```

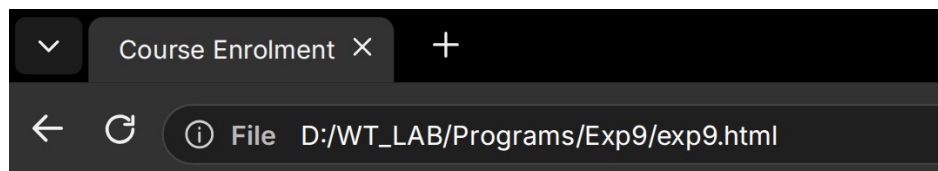
OUTPUT:**Validation errors for invalid input****Course Enrolment Form**

Name:
 Name must have at least 3 letters

Email:
 Enter a valid email

Age:
 Age must be between 17 and 30

Mobile:
 Mobile must have 10 digits

Valid form submitted**Course Enrolment Form**

Name:

Email:

Age:

Mobile:

Enrolled: Akhilesh Raj P, akhilesh.2401009@srec.ac.in, 20, 9876501234

RESULT:

Thus, an AngularJS enrolment form with field validation was created and tested successfully.

EX.NO.10	REACT JS: COMPONENTS, PROPS, STATE AND EVENTS
DATE: 28.09.26	

AIM:

To create a React application with reusable components, pass data through props, and use state and click events to update the page.

ALGORITHM:

1. Create a React project with Vite and open the src folder.
2. Create a ProfileCard component that receives name, role and skills as props.
3. Create a Counter component that stores the number of likes in state using useState.
4. Handle the click event of a button to increase the like count.
5. Use a second state value to toggle the availability status of the student.
6. Render both components inside the App component.
7. Run npm run dev and open the page in the browser.

CODE:**App.jsx:**

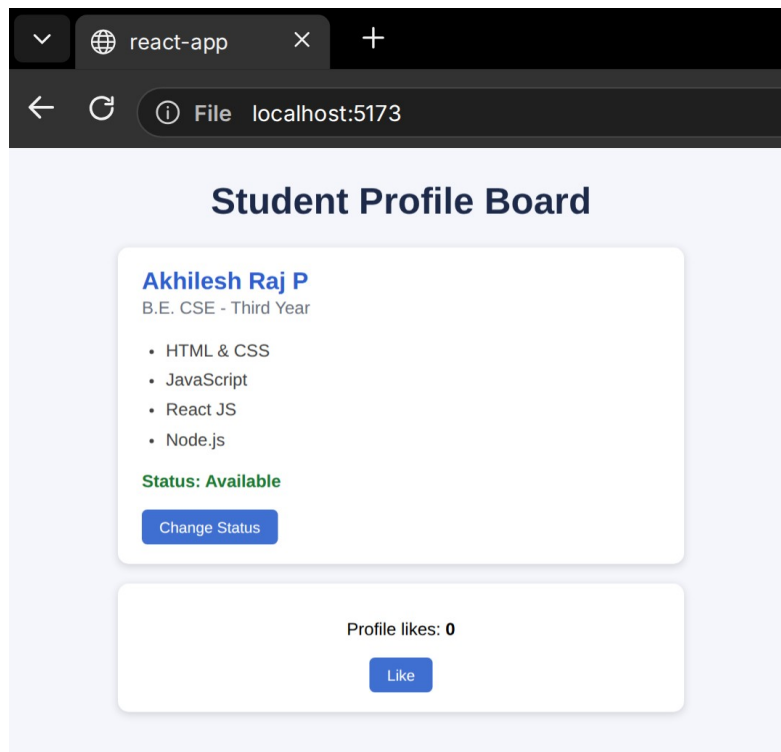
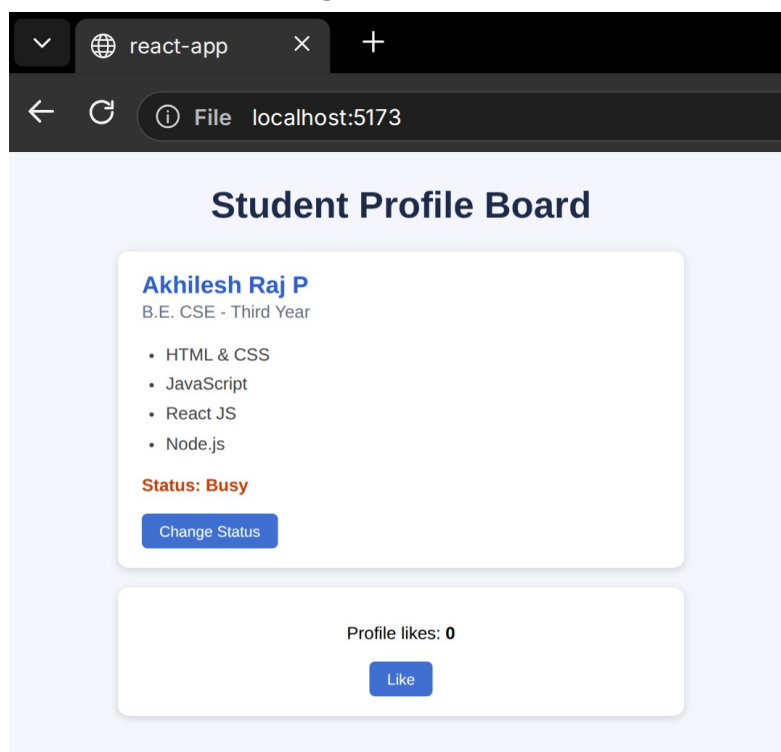
```
import { useState } from "react";
import "../App.css";

function ProfileCard({ name, role, skills }) {
  const [available, setAvailable] = useState(true);
  return (
    <div className="card">
      <h2>{name}</h2>
      <p className="role">{role}</p>
      <ul>
        {skills.map((s, i) => <li key={i}>{s}</li>)}
      </ul>
      <p className={available ? "on" : "off"}>
        Status: {available ? "Available" : "Busy"}
      </p>
      <button onClick={() => setAvailable(!available)}>Change Status</button>
    </div>
  );
}

function Counter() {
  const [likes, setLikes] = useState(0);
  return (
    <div className="card small">
      <p>Profile likes: <b>{likes}</b></p>
      <button onClick={() => setLikes(likes + 1)}>Like</button>
    </div>
  );
}

export default function App() {
  return (
    <div className="page">
      <h1>Student Profile Board</h1>
      <ProfileCard name="Akhilesh Raj P" role="B.E. CSE - Third Year"
        skills={["HTML & CSS", "JavaScript", "React JS", "Node.js"]} />
      <Counter />
    </div>
  );
}
```

}

OUTPUT:**Initial page****After Change Status and three likes****RESULT:**

Thus, a React application using components, props, state and events was developed successfully.

EX.NO.11**REACT JS: COMPONENT STATE AND INPUT HANDLING****DATE: 28.09.26****AIM:**

To build a feedback form in React that keeps every input in component state, shows a live preview and displays the submitted feedback.

ALGORITHM:

1. Create a React component named FeedbackForm.
2. Declare one state object holding name, email, course and rating.
3. Write a single handleChange function that updates the state from any input.
4. Connect each input to the state as a controlled component with value and onChange.
5. Show a live preview of the entered data below the form.
6. Validate the fields on submit and store the submitted data in a second state variable.
7. Display a confirmation card after a successful submit.

CODE:**FeedbackForm.jsx:**

```
import { useState } from "react";

export default function FeedbackForm() {
  const [form, setForm] = useState({ name: "", email: "", course: "", rating: "5" });
  const [sent, setSent] = useState(null);

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!form.name || !form.email || !form.course) {
      alert("Please fill all the fields");
      return;
    }
    setSent(form);
  };

  return (
    <div className="wrap">
      <h2>Course Feedback</h2>
      <form onSubmit={handleSubmit}>
        <label>Name</label>
        <input name="name" value={form.name} onChange={handleChange} />
        <label>Email</label>
        <input name="email" value={form.email} onChange={handleChange} />
        <label>Course</label>
        <select name="course" value={form.course} onChange={handleChange}>
          <option value="">Select</option>
          <option>Web Technologies</option>
          <option>Software Development Process</option>
          <option>Microcontrollers</option>
        </select>
        <label>Rating (1-5)</label>
        <input type="number" name="rating" min="1" max="5"
          value={form.rating} onChange={handleChange} />
        <button type="submit">Submit</button>
      </form>
      <p className="live">Live preview: {form.name} | {form.course} | {form.rating}/5</p>
    </div>
  );
}
```

```
    {sent && <div className="done">Thank you, {sent.name}! Feedback saved for {sent.cour  
se}</div>}  
    </div>  
  );  
}
```

OUTPUT:**Live preview while typing**

A screenshot of a web browser window titled 'react-app' at 'localhost:5173'. The page displays a 'Course Feedback' form. The form fields are: Name (Akhilesh Raj P), Email (akhilesh.2401009@srec.ac.in), Course (Web Technologies), and Rating (1-5) (5). A blue 'Submit' button is at the bottom. Below the form, a live preview shows: 'Live preview: Akhilesh Raj P | Web Technologies | 5/5'.

Feedback submitted

A screenshot of the same web browser window. The 'Course Feedback' form is shown with the same inputs as before, but the rating is now 4. Below the form, a green message box says: 'Thank you, Akhilesh Raj P! Feedback saved for Web Technologies.' The live preview now shows: 'Live preview: Akhilesh Raj P | Web Technologies | 4/5'.

RESULT:

Thus, a React feedback form with controlled inputs and live state updates was created successfully.

EX.NO.12	NODE.JS AND MONGODB
DATE: 28.09.26	

AIM:

To develop a Node.js and Express server connected to MongoDB that performs create, read, update and delete (CRUD) operations on library member records.

ALGORITHM:

1. Create a project folder and run `npm init -y`.
2. Install the packages `express` and `mongodb` with `npm install`.
3. Start the MongoDB server and open MongoDB Compass at `mongodb://127.0.0.1:27017`.
4. Create the database `library` and the collection `members`.
5. Write `server.js` that connects to MongoDB and enables JSON parsing in Express.
6. Create the routes `POST /members`, `GET /members`, `PUT /members/:id` and `DELETE /members/:id`.
7. Run `node server.js` and test every route from the browser page and Compass.

CODE:**server.js:**

```
const express = require("express");
const { MongoClient } = require("mongodb");

const app = express();
app.use(express.json());
app.use(express.static("public"));

const client = new MongoClient("mongodb://127.0.0.1:27017");
let members;

async function start() {
  await client.connect();
  members = client.db("library").collection("members");
  console.log("MongoDB connected - library.members");

  // CREATE
  app.post("/members", async (req, res) => {
    await members.insertOne(req.body);
    res.send("Member added successfully");
  });

  // READ
  app.get("/members", async (req, res) => {
    res.json(await members.find().toArray());
  });

  // UPDATE
  app.put("/members/:id", async (req, res) => {
    await members.updateOne({ memberId: req.params.id }, { $set: req.body });
    res.send("Member updated successfully");
  });

  // DELETE
  app.delete("/members/:id", async (req, res) => {
    await members.deleteOne({ memberId: req.params.id });
    res.send("Member deleted successfully");
  });
}
```

```
app.listen(4000, () => console.log("Server running at http://localhost:4000"));
}
start();
```

Sample document:

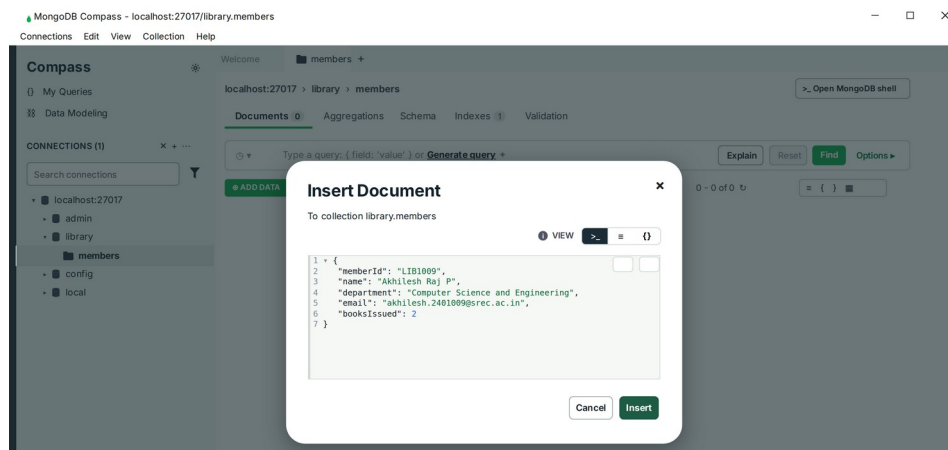
```
{
  "memberId": "LIB1009",
  "name": "Akhilesh Raj P",
  "department": "Computer Science and Engineering",
  "email": "akhilesh.2401009@srec.ac.in",
  "booksIssued": 2
}
```

OUTPUT:

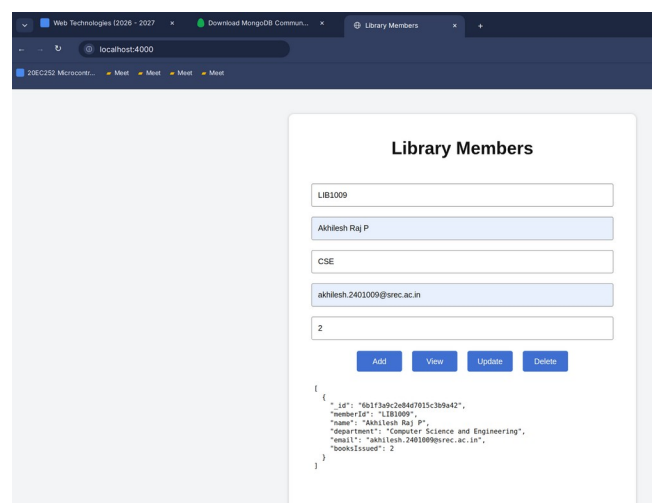
Server start

```
D:\WT_LAB\Programs\Exp12>node server.js
MongoDB connected - library.members
Server running at http://localhost:4000
```

ADD - insert document in MongoDB Compass



VIEW



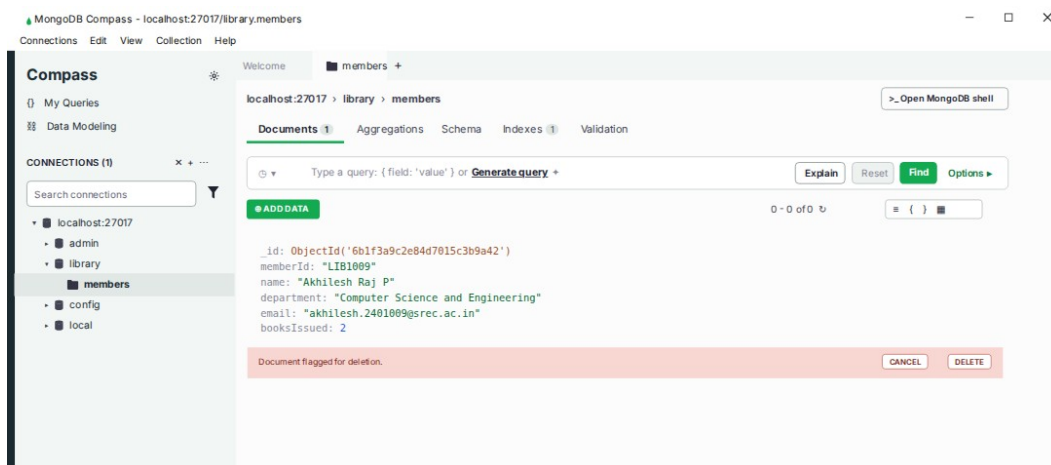
UPDATE

Library Members

AddViewUpdateDelete

Member updated successfully

DELETE - document flagged for deletion in Compass



RESULT:

Thus, a Node.js and MongoDB library member management system was developed and the CRUD operations were verified successfully.